

Inheritance is a technique for code reuse. It lets you create child classes that inherit the methods of their parent classes. You can override the methods to provide new code for them but also use the `super()` function to call the original methods in the parent class. A child class has an “is a” relationship with its parent class, because an object of the child class is a kind of object of the parent class.

In Python, using classes and inheritance is optional. Some programmers see the complexity that heavy use of inheritance creates as not worth its benefits. It’s often more flexible to use composition instead of inheritance, because it implements a “has a” relationship with an object of one class and objects of other classes rather than inheriting methods directly from those other classes. This means that objects of one class can have an object of another class. For example, a `Customer` object could have a `birthdate` method that is assigned a `Date` object rather than the `Customer` class subclassing `Date`.

Just as `type()` can return the type of the object passed to it, the `isinstance()` and `issubclass()` functions return type and inheritance information about the object passed to them.

Classes can have object methods and attributes, but they can also have class methods, class attributes, and static methods. Although these are rarely used, they can enable other object-oriented techniques that global variables and functions can’t provide.

Python lets classes inherit from multiple parents, although this can lead to code that is difficult to understand. The `super()` function and a class’s methods figure out how to inherit methods based on the MRO. You can view a class’s MRO in the interactive shell by calling the `mro()` method on the class.

This chapter and the previous one covered general OOP concepts. In the next chapter, we’ll explore Python-specific OOP techniques.